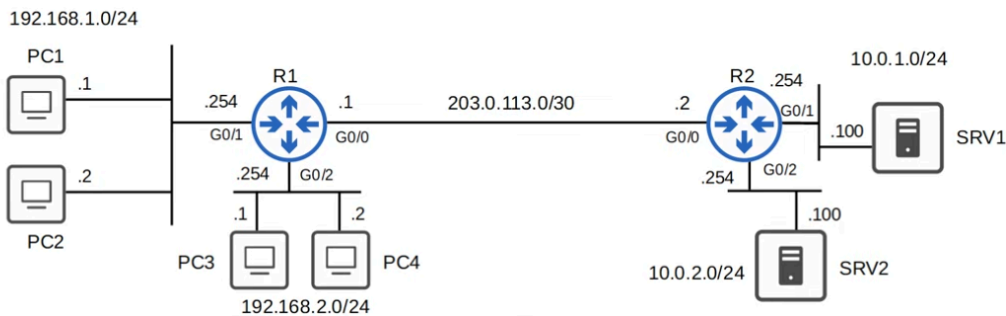




What are ACLs?

- ACLs (Access Control Lists) have multiple uses.
- In Day 34 and Day 35, we will focus on ACLs from a security perspective.
- ACLs function as a packet filter, instructing the router to permit or discard specific traffic.
- ACLs can filter traffic based on source/destination IP addresses, source/destination Layer 4 ports, etc.

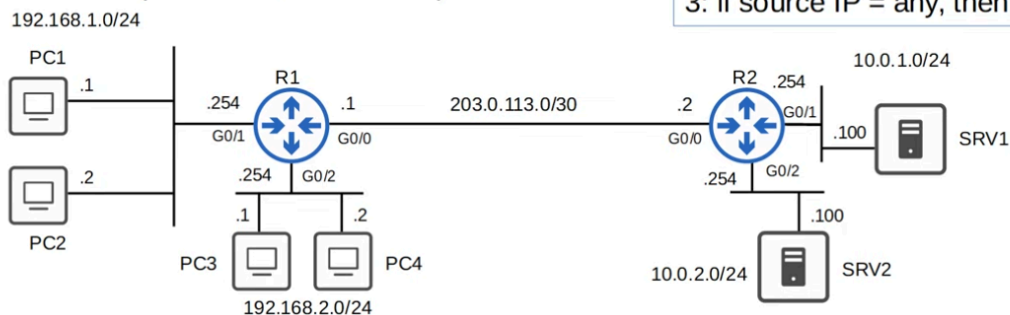


How ACLs work

- REQUIREMENT:
Hosts in 192.168.1.0/24 can access the 10.0.1.0/24 network
Hosts in 192.168.2.0/24 cannot access the 10.0.1.0/24 network.

- ACLs are configured globally on the router.
(global config mode)
- They are an ordered sequence of ACEs.
(Access Control Entries)

ACL 1:
1: if source IP = 192.168.1.0/24,
then permit
2: if source IP = 192.168.2.0/24,
then deny
3: if source IP = any, then permit

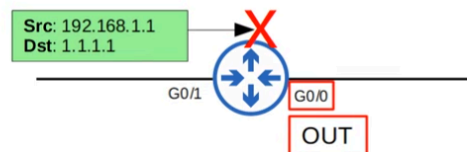




How ACLs work

- Configuring an ACL in global config mode will not make the ACL take effect.
- The ACL must be applied to an interface.
- ACLs are applied either inbound or outbound.
- ACLs are made up of one or more ACEs.
- When the router checks a packet against the ACL, it processes the ACEs in order, from top to bottom.
- If the packet matches one of the ACEs in the ACL, the router takes the action and stops processing the ACL. All entries below the matching entry will be ignored.

A maximum of one ACL can be applied to a single interface per direction.
Inbound: Maximum one ACL
Outbound: Maximum one ACL



ACL 2:
1: if source IP = 192.168.0.0/16,
then deny
~~2: if source IP = 192.168.1.0/24,
then permit~~



ACL Types

- Standard ACLs: Match based on **Source IP address** only
 - Standard Numbered ACLs
 - Standard Named ACLs

- Extended ACLs: Match based on **Source/Destination IP, Source/Destination port**, etc.
 - Extended Numbered ACLs
 - Extended Named ACLs



Standard Numbered ACLs

- Standard ACLs match traffic based only on the source IP address of the packet.
- Numbered ACLs are identified with a number (ie. ACL 1, ACL 2, etc)
- Different types of ACLs have a different range of numbers that can be used.
→ Standard ACLs can use 1-99 and 1300-1999.
- The basic command to configure a standard numbered ACL is:
R1(config)# **access-list** *number* {deny | permit} *ip wildcard-mask*

```
R1(config)# access-list 1 deny 1.1.1.1 0.0.0.0
R1(config)# access-list 1 deny 1.1.1.1
R1(config)# access-list 1 deny host 1.1.1.1
R1(config)# access-list 1 permit any
R1(config)# access-list 1 permit 0.0.0.0 255.255.255.255
R1(config)# access-list 1 remark ## BLOCK BOB FROM ACCOUNTING ##
```



Standard Numbered ACLs

```
R1(config)#access-list 1 deny 1.1.1.1 0.0.0.0
R1(config)#access-list 1 permit 0.0.0.0 255.255.255.255
R1(config)#access-list 1 remark ## BLOCK BOB FROM ACCOUNTING ##
R1(config)#
R1(config)#do show access-lists
Standard IP access list 1
  10 deny 1.1.1.1
  20 permit any
R1(config)#
R1(config)#do show ip access-lists
Standard IP access list 1
  10 deny 1.1.1.1
  20 permit any
R1(config)#
R1(config)#do show running-config | include access-list
access-list 1 deny 1.1.1.1
access-list 1 permit any
access-list 1 remark ## BLOCK BOB FROM ACCOUNTING ##
R1(config)#
```

Apply to an interface:

```
R1(config-if)# ip access-group number {in | out}
```

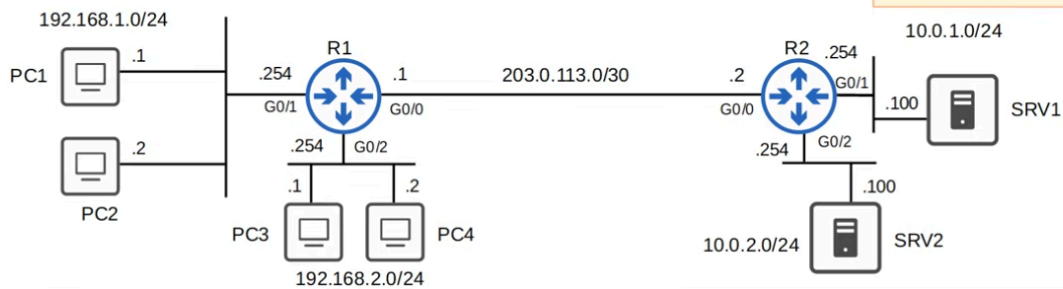




Standard Numbered ACLs

```
R1(config)#access-list 1 permit 192.168.1.1
R1(config)#access-list 1 deny 192.168.1.0 0.0.0.255
R1(config)#access-list 1 permit any
R1(config)#
R1(config)#interface g0/2
R1(config-if)#ip access-group 1 out
R1(config-if)#
```

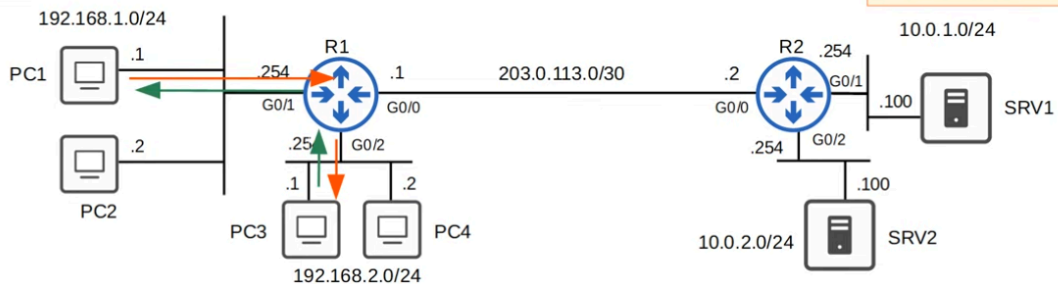
Standard ACLs should be applied as close to the destination as possible.



Standard Numbered ACLs

```
R1#show access-lists
Standard IP access list 1
10 permit 192.168.1.1
20 deny 192.168.1.0, wildcard bits 0.0.0.255
30 permit any
R1#
```

> ping 192.168.2.1





Standard Named ACLs

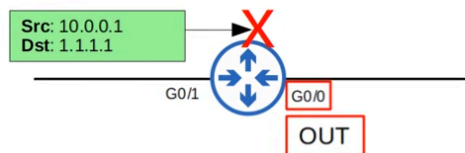
- Standard ACLs match traffic based only on the source IP address of the packet.
- Named ACLs are identified with a name (ie. 'BLOCK_BOB')
- Standard named ACLs are configured by entering 'standard named ACL config mode', and then configuring each entry within that config mode.
R1(config)# **ip access-list standard** *acl-name*
R1(config-std-nacl)# [*entry-number*] {deny | permit} *ip wildcard-mask*

```
R1(config)#ip access-list standard BLOCK_BOB
R1(config-std-nacl)#5 deny 1.1.1.1
R1(config-std-nacl)#10 permit any
R1(config-std-nacl)#remark ## CONFIGURED NOV 21 2020 ##
R1(config-std-nacl)#interface g0/0
R1(config-if)#ip access-group BLOCK_BOB in
```



Implicit deny

- What will happen if a packet doesn't match any of the entries in an ACL?



ACL 2:
1: if source IP = 192.168.1.0/24,
then permit
2: if source IP = 192.168.0.0/16,
then deny
(3: if source IP = any, then deny)

- There is an 'implicit deny' at the end of all ACLs.
- The implicit deny tells the router to deny all traffic that doesn't match any of the configured entries in the ACL.

Standard Named ACLs

```
R1#show access-lists
Standard IP access list BLOCK_BOB
 5 deny 1.1.1.1
10 permit any
R1#
R1#show running-config | section access-list
ip access-list standard BLOCK_BOB
deny 1.1.1.1
permit any
remark ## CONFIGURED NOV 21 2020 ##
R1#
```

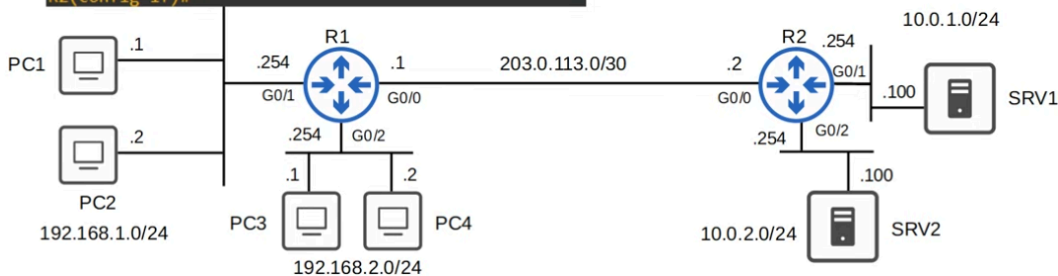


Standard Named ACLs

```
R2(config)#ip access-list standard T0_10.0.2.0/24
R2(config-std-nacl)#deny 192.168.1.0 0.0.0.255
R2(config-std-nacl)#permit any
R2(config-std-nacl)#interface g0/2
R2(config-if)#ip access-group T0_10.0.2.0/24 out
R2(config-if)#
R2(config-if)#ip access-list standard T0_10.0.1.0/24
R2(config-std-nacl)#deny 192.168.2.1
R2(config-std-nacl)#permit 192.168.2.0 0.0.0.255
R2(config-std-nacl)#permit 192.168.1.1
R2(config-std-nacl)#deny 192.168.1.0 0.0.0.255
R2(config-std-nacl)#permit any
R2(config-std-nacl)#interface g0/1
R2(config-if)#ip access-group T0_10.0.1.0/24 out
R2(config-if)#
```

Requirements:

- PCs in 192.168.1.0/24 can't access 10.0.2.0/24.
- PC3 can't access 10.0.1.0/24.
- Other PCs in 192.168.2.0/24 can access 10.0.1.0/24.
- PC1 can access 10.0.1.0/24.
- Other PCs in 192.168.1.0/24 can't access 10.0.1.0/24.



ACCESS CONTROL LIST (ACL) — REVISION NOTES

1. What is an ACL?

ACL = Access Control List

An ACL is a set of rules used by a router to **permit or deny network traffic**.

Think of it as a **security filter/gate**:

Packet → ACL checks packet → PERMIT / DENY

ACLs can filter traffic based on:

- Source IP address
 - Destination IP address
 - Source Layer 4 port
 - Destination Layer 4 port
 - Protocol such as TCP/UDP/ICMP
-

2. ACL Types

There are two major ACL types:

Standard ACL

Checks **only the source IP address**.

Source IP → Permit/Deny

Extended ACL

Can check:

Source IP
Destination IP
Protocol
Source port
Destination port

Quick comparison

Feature	Standard ACL	Extended ACL
Source IP	✓	✓
Destination IP	✗	✓
Protocol	✗	✓
Port number	✗	✓
Typical placement	Near destination	Near source

3. ACL Structure

An ACL consists of one or more **ACEs**.

ACE

ACE = Access Control Entry

Each ACE is one individual rule.

Example:

ACL 1

1. permit 192.168.1.0/24
2. deny 192.168.2.0/24
3. permit any

Here there are **3 ACEs** inside ACL 1.

Think:

ACL

- ├── ACE 1
 - ├── ACE 2
 - └── ACE 3
-

4. ACLs Are Configured Globally

The ACL itself is created in:

Global Configuration Mode

Example:

```
R1(config)# access-list 1 deny host 1.1.1.1
```

But creating the ACL **does not make it active**.

The ACL must be **applied to an interface**.

Example:

```
R1(config)# interface g0/0  
R1(config-if)# ip access-group 1 in
```

5. Inbound vs Outbound

ACLs can be applied in two directions:

IN

The packet is checked when it **enters the router interface**.

Packet

↓

G0/0 → [ACL IN] → Router

OUT

The packet is checked when it **leaves the router interface**.

Router → [ACL OUT] → G0/1

Command:

ip access-group <ACL> in

or

ip access-group <ACL> out

6. Important Interface Rule

A single interface can have:

Maximum 1 ACL inbound

Maximum 1 ACL outbound

So theoretically:

Interface G0/0

IN → 1 ACL maximum

OUT → 1 ACL maximum

Therefore, an interface can have **one inbound ACL and one outbound ACL**.

7. How an ACL Processes Packets

ACL entries are processed:

TOP → BOTTOM

The router checks the packet against each ACE.

As soon as a packet matches an ACE:

MATCH

↓

Take the action

↓

STOP checking the ACL

All entries below the matching entry are ignored.

Example

ACL 2

1. deny 192.168.0.0/16
2. permit 192.168.1.0/24

A packet from:

192.168.1.10

matches the **first rule** because:

192.168.1.10 belongs to 192.168.0.0/16

Therefore:

DENY ❌

The router **never reaches rule 2**.

Important:

First matching ACE wins.

8. Implicit Deny

Every ACL has an invisible rule at the end:

deny any

This is called:

Implicit Deny

Example:

ACL 2

1. permit 192.168.1.0/24
2. deny 192.168.0.0/16
3. deny any ← implicit

You don't normally see the third rule in the configuration, but it exists logically.

Therefore:

If a packet doesn't match any configured ACE, it is denied.

9. How to Allow Everyone Else

Suppose you have:

```
access-list 1 deny host 1.1.1.1
```

You might think:

Only 1.1.1.1 will be denied.

But because of implicit deny:

1.1.1.1 → DENY
Everyone else → DENY

To allow everyone else, add:

```
access-list 1 permit any
```

Now:

1.1.1.1 →  DENY

Everyone else →  PERMIT

10. Standard Numbered ACL

A **numbered ACL** is identified using a number.

For Standard ACLs:

1–99

and:

1300–1999

are the traditional standard numbered ACL ranges.

Important

The ACL number is an **identifier**.

It is **NOT a priority**.

For example:

ACL 10

ACL 20

ACL 30

The number simply identifies the ACL.

It is not like:

10 = high priority

20 = medium priority

30 = low priority

 Wrong.

The **order of ACEs inside an ACL** determines which rule is checked first.

11. Standard Numbered ACL Syntax

Basic syntax:

```
access-list <number> {deny | permit} <IP> <wildcard-mask>
```

Example:

```
access-list 1 deny 1.1.1.1 0.0.0.0
```

This denies exactly:

```
1.1.1.1
```

12. Three Ways to Specify a Single Host

These are equivalent:

Method 1

```
access-list 1 deny 1.1.1.1 0.0.0.0
```

Method 2

```
access-list 1 deny 1.1.1.1
```

Method 3

```
access-list 1 deny host 1.1.1.1
```

The **host** keyword means:

Match exactly this one IP address.

13. any

any means:

Any source IP address.

Example:

```
access-list 1 permit any
```

Equivalent conceptually to:

```
access-list 1 permit 0.0.0.0 255.255.255.255
```

So:

any = every IPv4 address

14. Wildcard Mask

ACLs commonly use a **wildcard mask**.

Remember:

0 → must match

1 → don't care

Single host

```
1.1.1.1 0.0.0.0
```

Every bit must match.

/24 network

```
192.168.1.0 0.0.0.255
```

This matches:

192.168.1.0 – 192.168.1.255

So:

```
access-list 1 deny 192.168.1.0 0.0.0.255
```

means:

Deny the entire **192.168.1.0/24** network.

15. remark

remark is used to add a **comment/description** to an ACL.

Example:

```
access-list 1 remark ## BLOCK BOB FROM ACCOUNTING ##
```

It does not affect packet filtering.

It is simply there to help administrators understand the purpose of the ACL.

16. Applying a Numbered ACL

Creating:

```
access-list 1 deny host 1.1.1.1  
access-list 1 permit any
```

doesn't activate it.

Apply it:

```
interface g0/0
ip access-group 1 in
```

Meaning:

Apply ACL 1 to traffic entering G0/0.

For outbound:

```
interface g0/0
ip access-group 1 out
```

17. Standard ACL Placement

Main rule:

Standard ACLs should generally be placed as close to the destination as possible.

Why?

Because Standard ACL only checks:

SOURCE IP

It doesn't know the destination.

Example

Suppose:

PC1 — R1 — R2 — Server

PC1:

192.168.1.10

Server:

10.0.1.100

Requirement:

Block PC1 from reaching the Server.

If the Standard ACL is placed close to PC1:

PC1 — [ACL] — R1 — R2 — Server

The ACL says:

Source = 192.168.1.10 → DENY

But Standard ACL doesn't know **where PC1 is going**.

So it may block PC1 from reaching:

Server A ✗
Server B ✗
Server C ✗
Internet ✗

If the ACL is placed close to the destination:

PC1 — R1 — R2 — [ACL] — Server

then the restriction is applied near the destination.

This reduces the chance of unnecessarily blocking the source from reaching other destinations.

18. Extended ACL Placement

Extended ACL can check:

Source

Destination
Protocol
Port

Therefore, it can make a much more specific decision.

Example:

```
access-list 100 deny tcp host 192.168.1.10 host 10.0.1.100 eq 80
```

This means:

Source: 192.168.1.10
Destination: 10.0.1.100
Protocol: TCP
Port: 80
Action: DENY

Only that specific traffic is denied.

Therefore:

Extended ACLs are generally placed as close to the source as possible.

19. Standard Numbered ACL Example

Suppose:

PC1 = 192.168.1.1
PC2 = 192.168.1.2
PC3 = 192.168.2.1
PC4 = 192.168.2.2

Requirement:

PC1 → can access 192.168.2.0/24
Other PCs in 192.168.1.0/24 → cannot access 192.168.2.0/24

ACL:

```
access-list 1 permit host 192.168.1.1  
access-list 1 deny 192.168.1.0 0.0.0.255  
access-list 1 permit any
```

Why this order?

First:

```
permit host 192.168.1.1
```

allows PC1.

Then:

```
deny 192.168.1.0 0.0.0.255
```

blocks the other PCs in that network.

Finally:

```
permit any
```

allows everyone else.

20. Why Sequence/Entry Numbers Matter

In a named ACL, you can assign sequence numbers:

```
5 deny 1.1.1.1  
10 permit any
```

The sequence number identifies the position of the ACE.

Example:

```
5 deny 1.1.1.1
```

10 permit any

The router logically processes the ACEs in their appropriate order.

Important distinction

ACL number:

ACL 1
ACL 10
ACL 100

→ Identifies the ACL.

ACE sequence number:

5
10
20
30

→ Identifies/organizes individual entries inside the ACL.

21. Standard Named ACL

Instead of using a number:

```
access-list 1
```

you can give the ACL a meaningful name.

Example:

```
ip access-list standard BLOCK_BOB
```

Now you're inside Standard Named ACL configuration mode:

```
R1(config-std-nacl)#
```

Then add entries:

```
deny host 1.1.1.1  
permit any
```

Complete example:

```
R1(config)# ip access-list standard BLOCK_BOB  
R1(config-std-nacl)# 5 deny host 1.1.1.1  
R1(config-std-nacl)# 10 permit any
```

Apply it:

```
R1(config)# interface g0/0  
R1(config-if)# ip access-group BLOCK_BOB in
```

22. Numbered vs Named ACL

Feature	Numbered ACL	Named ACL
Identifier	Number	Name
Example	ACL 1	BLOCK_BOB
Easy to understand	Less descriptive	More descriptive
Sequence numbers	Supported	Supported
Standard	Yes	Yes
Extended	Yes	Yes

Example:

Numbered:
access-list 1 deny host 1.1.1.1

Named:

```
ip access-list standard BLOCK_BOB  
deny host 1.1.1.1
```

23. Named ACL Example from the Topology

Suppose:

192.168.1.0/24 → PCs
192.168.2.0/24 → PCs

10.0.1.0/24 → SRV1
10.0.2.0/24 → SRV2

Requirement:

192.168.1.0/24 → cannot access 10.0.2.0/24

192.168.2.0/24 → can access 10.0.2.0/24

Create:

```
ip access-list standard TO_10.0.2.0/24  
deny 192.168.1.0 0.0.0.255  
permit any
```

Then apply it to the interface leading toward:

10.0.2.0/24

For example:

```
interface g0/2  
ip access-group TO_10.0.2.0/24 out
```

This is an example of placing a **Standard ACL** close to the destination.

24. ACL Verification Commands

Show all ACLs

```
show access-lists
```

or:

```
show ip access-lists
```

Example:

```
Standard IP access list 1
```

```
10 deny 1.1.1.1
```

```
20 permit any
```

Show ACL configuration

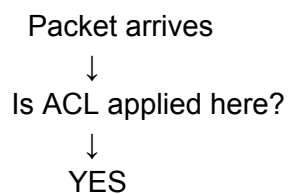
```
show running-config | include access-list
```

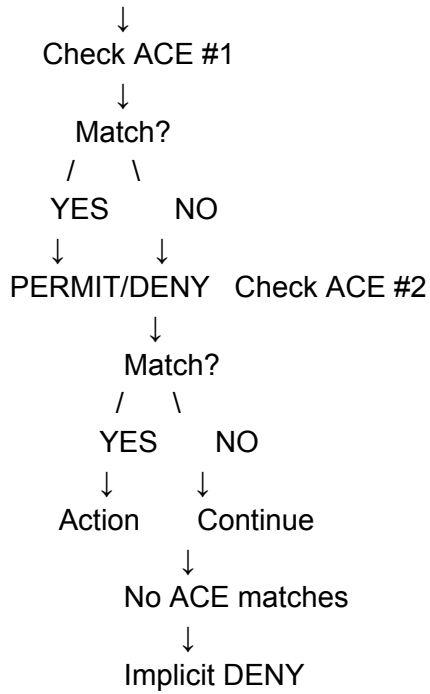
For named ACL sections:

```
show running-config | section access-list
```

25. Complete ACL Packet-Processing Flow

Remember this flow:





★ 26. Most Important Exam Points

ACL basics

ACL = Access Control List
ACE = Access Control Entry

Standard ACL

Source IP only

Extended ACL

Source IP
Destination IP
Protocol
Ports

Number ranges

Standard: 1–99

Standard: 1300–1999

Extended: 100–199

Extended: 2000–2699

ACL number

Identifier, not priority.

ACE processing

Top → bottom

Matching

First matching ACE wins.

No match

Implicit deny

Creating ACL

Done in **global configuration mode**.

Applying ACL

Must be applied to an **interface**.

Direction

in → entering interface

out → leaving interface

Interface limit

1 inbound ACL

1 outbound ACL

Placement

Standard ACL → close to destination

Extended ACL → close to source

host

host 1.1.1.1

→ one specific IP.

any

any

→ any IP.

Wildcard mask

0 = must match

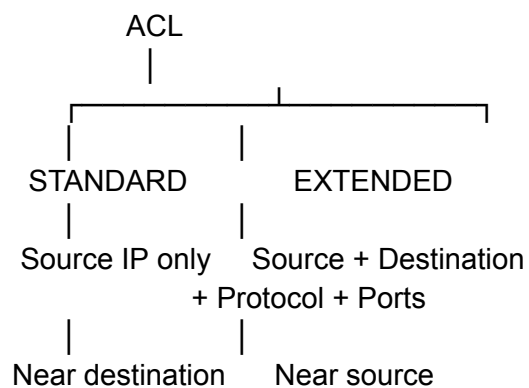
1 = don't care

remark

Comment/description; doesn't affect filtering.

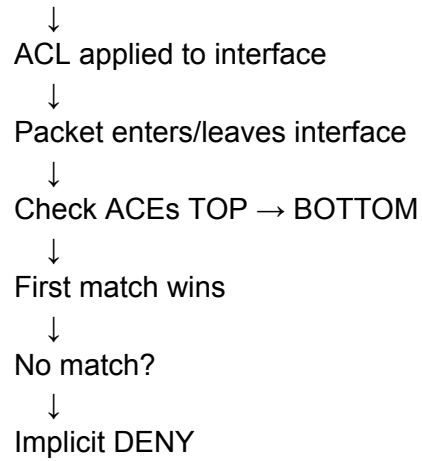


ONE-MINUTE ACL REVISION



And the packet-processing rules:

ACL created



🔥 Final memory trick

Standard = WHO? → Source IP

Extended = WHO + WHERE + WHAT? → Source + Destination + Protocol/Port

Number = ACL identity

Sequence = ACE position

First match = final decision

No match = implicit deny

Standard → destination

Extended → source

🔑 ACL — Advanced Notes



Configuring numbered ACLs with subcommands

- In Day 34, you learned that numbered ACLs are configured in global config mode:

```
R1(config)# access-list 1 deny 192.168.1.1
R1(config)# access-list 1 permit any
```
- You learned that named ACLs are configured with subcommands in a separate config mode:

```
R1(config)# ip access-list standard BLOCK_PC1
R1(config-std-nacl)# deny 192.168.1.1
R1(config-std-nacl)# permit any
```
- However, in modern IOS you can also configure numbered ACLs in the exact same way as named ACLs:

```
R1(config)# ip access-list standard 1
R1(config-std-nacl)# deny 192.168.1.1
R1(config-std-nacl)# permit any
```
- This is just a different way of configuring numbered ACLs. However, in the running-config the ACL will display as if it was configured using the traditional method.



Configuring numbered ACLs with subcommands

```
R1(config)#ip access-list standard ?
<1-99>      Standard IP access-list number
<1300-1999> Standard IP access-list number (expanded range)
WORD        Access-list name

R1(config)#ip access-list standard 1
R1(config-std-nacl)#deny 192.168.1.1
R1(config-std-nacl)#permit any
R1(config-std-nacl)#
R1(config-std-nacl)#do show running-config | section access-list
access-list 1 deny 192.168.1.1
access-list 1 permit any
R1(config-std-nacl)#
```

1. Numbered ACL with Subcommands

Normally, a numbered Standard ACL can be created directly in global configuration mode:

```
R1(config)# access-list 1 deny 192.168.1.1
R1(config)# access-list 1 permit any
```

But modern Cisco IOS also allows a numbered ACL to be configured using a separate ACL configuration mode:

```
R1(config)# ip access-list standard 1
R1(config-std-nacl)# deny 192.168.1.1
R1(config-std-nacl)# permit any
```

Important

Both methods create the **same ACL**.

If you check:

```
R1# show running-config | section access-list
```

you may see:

```
access-list 1 deny 192.168.1.1  
access-list 1 permit any
```

So:

Configuration method is different, but the resulting ACL is the same.

2. Why use ACL Configuration Mode?



Advantages of named ACL config mode

- You can easily delete individual entries in the ACL with **no entry-number**.

```
R1(config-std-nacl)#do show access-lists  
Standard IP access list 1  
 10 deny 192.168.1.1  
 20 deny 192.168.1.2  
 30 deny 192.168.3.0, wildcard bits 0.0.0.255  
 40 permit any  
R1(config-std-nacl)#  
R1(config-std-nacl)#no 30  
R1(config-std-nacl)#  
R1(config-std-nacl)#do show access-lists  
Standard IP access list 1  
 10 deny 192.168.1.1  
 20 deny 192.168.1.2  
 40 permit any  
R1(config-std-nacl)#
```

The biggest advantage is **easy editing**.

Suppose ACL 1 contains:

```
10 deny 192.168.1.1  
20 deny 192.168.1.2  
30 deny 192.168.3.0 0.0.0.255  
40 permit any
```

With ACL configuration mode:

```
R1(config)# ip access-list standard 1  
R1(config-std-nacl)#
```

You can delete only entry 30:

```
R1(config-std-nacl)# no 30
```

Only this rule disappears.

Traditional method

Advantages of named ACL config mode

```
R1(config)#do show access-lists
Standard IP access list 1
 10 deny 192.168.1.1
 20 deny 192.168.1.2
 30 deny 192.168.3.0, wildcard bits 0.0.0.255
 40 permit any
R1(config)#do show running-config | section access-list
access-list 1 deny 192.168.1.1
access-list 1 deny 192.168.1.2
access-list 1 deny 192.168.3.0 0.0.0.255
access-list 1 permit any
R1(config)#no access-list 1 deny 192.168.3.0 0.0.0.255
R1(config)#do show access-lists
R1(config)#do show running-config | section access-list
R1(config)#
```

When configuring/editing numbered ACLs from global config mode, you can't delete individual entries, you can only delete the entire ACL!

If you configure numbered ACL entries directly from global configuration:

```
R1(config)# access-list 1 deny 192.168.1.1
R1(config)# access-list 1 deny 192.168.1.2
R1(config)# access-list 1 deny 192.168.3.0 0.0.0.255
R1(config)# access-list 1 permit any
```

you **cannot simply remove an individual entry using its sequence number.**

You generally have to remove the entire ACL:

```
R1(config)# no access-list 1
```

⚠ This deletes the **whole ACL**.

3. Sequence Numbers



Advantages of named ACL config mode

- You can easily delete individual entries in the ACL with **no** *sequence-number*.
- You can insert new entries in between other entries by specifying the sequence number.

```
R1(config-std-nacl)#do show access-lists
Standard IP access list 1
 10 deny 192.168.1.1
 20 deny 192.168.1.2
 40 permit any
R1(config-std-nacl)#
R1(config-std-nacl)#30 deny 192.168.2.0 0.0.0.255
R1(config-std-nacl)#
R1(config-std-nacl)#do show access-lists
Standard IP access list 1
 10 deny 192.168.1.1
 20 deny 192.168.1.2
 30 deny 192.168.2.0, wildcard bits 0.0.0.255
 40 permit any
R1(config-std-nacl)#
R1(config-std-nacl)#do show running-config | section access-list
access-list 1 deny 192.168.1.1
access-list 1 deny 192.168.1.2
access-list 1 deny 192.168.2.0 0.0.0.255
access-list 1 permit any
```



Sequence numbers tell us the **order in which ACL entries are checked**.

Example:

```
10 deny 192.168.1.1
20 deny 192.168.1.2
30 deny 192.168.2.0 0.0.0.255
40 permit any
```

The router checks:

10 → 20 → 30 → 40

Why sequence numbers are useful?

You can insert a new rule between existing rules.

For example:

```
10 deny 192.168.1.1
20 deny 192.168.1.2
40 permit any
```

You want to add:

```
30 deny 192.168.2.0 0.0.0.255
```

Simply:

```
R1(config-std-nacl)# 30 deny 192.168.2.0 0.0.0.255
```

Now:

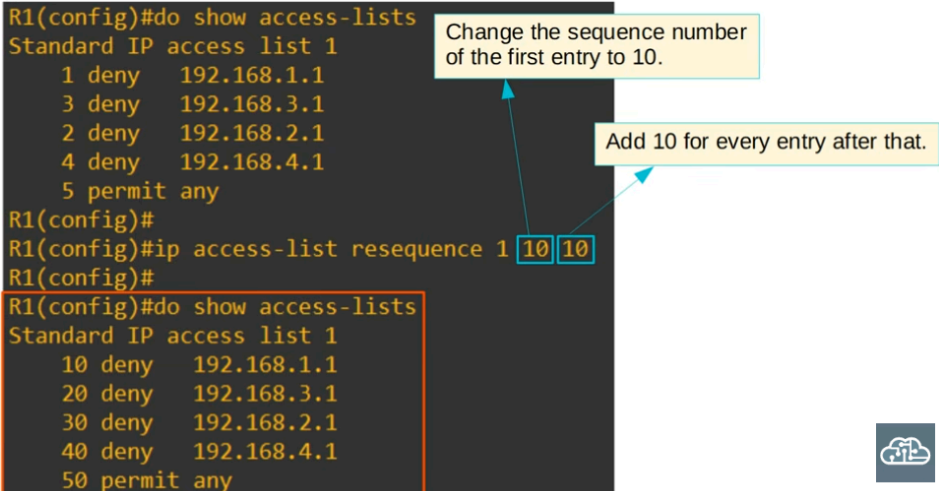
```
10 deny 192.168.1.1
20 deny 192.168.1.2
30 deny 192.168.2.0 0.0.0.255
40 permit any
```

4. Resequencing an ACL



Resequencing ACLs

- There is a *resequencing* function that helps edit ACLs.
- The command is **ip access-list resequence *acl-id starting-seq-num increment***



```
R1(config)#do show access-lists
Standard IP access list 1
 1 deny 192.168.1.1
 3 deny 192.168.3.1
 2 deny 192.168.2.1
 4 deny 192.168.4.1
 5 permit any
R1(config)#
R1(config)#ip access-list resequence 1 10 10
R1(config)#
R1(config)#do show access-lists
Standard IP access list 1
10 deny 192.168.1.1
20 deny 192.168.3.1
30 deny 192.168.2.1
40 deny 192.168.4.1
50 permit any
```

Change the sequence number of the first entry to 10.

Add 10 for every entry after that.

Sometimes sequence numbers become messy:

```
1 deny 192.168.1.1
3 deny 192.168.3.1
2 deny 192.168.2.1
4 deny 192.168.4.1
5 permit any
```

You can reorganize them using:

```
R1(config)# ip access-list resequence 1 10 10
```

Format:

ip access-list resequence ACL-ID starting-sequence increment

Here:

- 1 → ACL number
- 10 → starting sequence number
- 10 → increment

Result:

```
10 deny 192.168.1.1
20 deny 192.168.3.1
30 deny 192.168.2.1
40 deny 192.168.4.1
50 permit any
```

Remember

starting number = 10

increment = 10

So every next entry increases by 10.



5. Extended ACL



Extended ACLs

- Extended ACLs function mostly the same as standard ACLs.
- They can be numbered or named, just like standard ACLs.
→ Numbered ACLs use the following ranges: 100 – 199, 2000 – 2699
- They are processed from top to bottom, just like standard ACLs.
- However, they can match traffic based on more parameters, so they are more precise (and more complex) than standard ACLs.
- We will focus on matching based on these main parameters: **Layer 4 protocol/port**, **source address**, and **destination address**.

```
R1(config)# access-list number [permit | deny] protocol src-ip dest-ip
```

```
R1(config)# ip access-list extended {name | number}
R1(config-ext-nacl)# [seq-num] [permit | deny] protocol src-ip dest-ip
```



Standard ACL mainly checks:

Source IP address

Extended ACL can check much more.

It can match:

- Source IP
- Destination IP
- Protocol
- TCP/UDP source port
- TCP/UDP destination port
- Other parameters

So extended ACL gives **much more precise control**.

Number ranges

Extended numbered ACLs use:

100–199

2000–2699

Standard ACLs use:

1–99

1300–1999

6. Standard vs Extended ACL

Feature	Standard ACL	Extended ACL
Source IP	✓	✓
Destination IP	✗	✓
Protocol	✗	✓
TCP/UDP port	✗	✓
Precision	Lower	Higher

Numbered	1–99,	100–199,
range	1300–1999	2000–2699

Easy memory

Standard = WHO is sending?

Extended = WHO → WHERE → WHAT → WHICH PORT?

7. Extended ACL Syntax

Named extended ACL:

```
R1(config)# ip access-list extended HTTP_SRV1
R1(config-ext-nacl)# deny tcp 192.168.1.0 0.0.0.255 host 10.0.1.100 eq 443
R1(config-ext-nacl)# permit ip any any
```

Break it down:

```
deny
↓
tcp
↓
source network
↓
destination host
↓
destination port
```

So:

```
deny tcp 192.168.1.0 0.0.0.255 host 10.0.1.100 eq 443
```

means:

Deny TCP traffic from **192.168.1.0/24** to **10.0.1.100**, when the destination port is **443 (HTTPS)**.

8. Matching the Protocol

Matching the protocol

```
R1(config)#ip access-list extended EXAMPLE
R1(config-ext-nacl)#deny ?
<0-255>      An IP protocol number
ahp          Authentication Header Protocol
eigrp        Cisco's EIGRP routing protocol
esp          Encapsulation Security Payload
gre          Cisco's GRE tunneling
icmp         Internet Control Message Protocol
igmp         Internet Gateway Message Protocol
ip           Any Internet Protocol
ipinip       IP in IP tunneling
nos          KA9Q NOS compatible IP over IP tunneling
object-group Service object group
ospf         OSPF routing protocol
pcp          Payload Compression Protocol
pim          Protocol Independent Multicast
sctp         Stream Control Transmission Protocol
tcp          Transmission Control Protocol
udp          User Datagram Protocol
```

Extended ACL can identify the IP protocol.

Common options:

tcp
udp
icmp
ip
ospf
eigrp
gre
esp
ahp

Most important

tcp

Matches TCP traffic.

Examples:

- HTTP
- HTTPS
- SSH

- Telnet
- FTP

udp

Matches UDP traffic.

Examples:

- DNS
- DHCP
- TFTP

icmp

Used for ICMP traffic such as:

ping

ip

Means:

Any IP protocol

Example:

permit ip any any

means allow any IP traffic.

9. Source and Destination IP



Matching the source/destination IP address

```
R1(config-ext-nacl)#deny tcp ?
A.B.C.D      Source address
any          Any source host
host         A single source host
object-group Source network object group

R1(config-ext-nacl)#deny tcp any ?
A.B.C.D      Destination address
any          Any destination host
eq           Match only packets on a given port number
gt           Match only packets with a greater port number
host         A single destination host
lt           Match only packets with a lower port number
neq          Match only packets not on a given port number
object-group Destination network object group
range        Match only packets in the range of port numbers

R1(config-ext-nacl)#deny tcp any 10.0.0.0 ?
A.B.C.D      Destination wildcard bits

R1(config-ext-nacl)#deny tcp any 10.0.0.0 0.0.0.255
R1(config-ext-nacl)#
```

In extended ACLs, to specify a /32 source or destination you have to use the **host** option or specify the wildcard mask. You can't just write the address without either of those.



Matching the source/destination IP address

```
R1(config-ext-nacl)#deny tcp ?
A.B.C.D      Source address
any          Any source host
host         A single source host
object-group Source network object group

R1(config-ext-nacl)#deny tcp any ?
A.B.C.D      Destination address
any          Any destination host
eq           Match only packets on a given port number
gt           Match only packets with a greater port number
host         A single destination host
lt           Match only packets with a lower port number
neq          Match only packets not on a given port number
object-group Destination network object group
range        Match only packets in the range of port numbers

R1(config-ext-nacl)#deny tcp any 10.0.0.0 ?
A.B.C.D      Destination wildcard bits

R1(config-ext-nacl)#deny tcp any 10.0.0.0 0.0.0.255
R1(config-ext-nacl)#
```



1. Allow all traffic

```
R1(config-ext-nacl)#permit ip any any
```

2. Prevent 10.0.0.0/16 from sending UDP traffic to 192.168.1.1/32

```
R1(config-ext-nacl)#deny udp 10.0.0.0 0.0.255.255 host 192.168.1.1
```

3. Prevent 172.16.1.1/32 from pinging hosts in 192.168.0.0/24

```
R1(config-ext-nacl)#deny icmp host 172.16.1.1 192.168.0.0 0.0.0.255
```



Extended ACL follows this order:

SOURCE → DESTINATION

Example:

```
deny tcp 192.168.1.0 0.0.0.255 10.0.0.0 0.0.0.255
```

Means:

Source:

192.168.1.0/24



Destination:

10.0.0.0/24

any

any

means:

Any IP address.

Example:

```
deny tcp any 10.0.0.0 0.0.0.255
```

Means:

Deny TCP traffic from **any source** to the 10.0.0.0/24 network.

10. **host** Keyword

If you want to specify **one single IP address**, use:

```
host 10.0.0.100
```

Example:

```
deny tcp any host 10.0.0.100
```

Means:

Deny TCP traffic from any source to exactly 10.0.0.100.

Important

A /32 host can be represented by:

host 10.0.0.100

or:

10.0.0.100 0.0.0.0

But in extended ACL syntax, you cannot simply write:

deny tcp any 10.0.0.100

You normally use:

host 10.0.0.100



11. TCP/UDP Port Matching



Matching the TCP/UDP port numbers

- When matching TCP/UDP, you can optionally specify the source and/or destination port numbers to match.

```
R1(config-ext-nacl)#deny tcp src-ip eq src-port-num dest-ip eq dst-port-num
                             gt
                             lt
                             neq
                             range
                             neq
                             range
```

- eq 80 = equal to port 80
- gt 80 = greater than 80 (81 and greater)
- lt 80 = less than 80 (79 and less)
- neq 80 = NOT 80
- range 80 100 = from port 80 to port 100

TCP

- FTP data (20)
- FTP control (21)
- SSH (22)
- Telnet (23)
- SMTP (25)
- HTTP (80)
- POP3 (110)
- HTTPS (443)

UDP

- DHCP server (67)
- DHCP client (68)
- TFTP (69)
- SNMP agent (161)
- SNMP manager (162)
- Syslog (514)

TCP & UDP

- DNS (53)



This is one of the **most important extended ACL concepts**.

You can filter based on TCP/UDP port numbers.

Syntax:

eq
gt
lt
neq
range

eq

Equal to

eq 80

Means:

Exactly port 80.

Example:

deny tcp any host 10.0.0.100 eq 80

Blocks HTTP traffic to that server.

gt

Greater than

gt 80

Means:

81, 82, 83, ...

lt

Less than

It 80

Means:

79, 78, 77, ...

neq

Not equal to

neq 80

Means:

Every port except port 80.

range

Used for a range of ports.

range 80 100

Means:

80 through 100



Matching the TCP/UDP port numbers

```
R1(config-ext-nacl)#deny tcp any host 1.1.1.1 eq ?
0:65535
Port number
bgp      Border Gateway Protocol (179)
chargen  Character generator (19)
cmd      Remote commands (rcmd, 514)
daytime  Daytime (13)
discard  Discard (0)
domain  Domain Name Service (53)
drip     Dynamic Routing Information Protocol (3949)
echo     Echo (2)
exec     Exec (rsh, 512)
finger   Finger (79)
ftp      File Transfer Protocol (21)
ftp-data FTP data connections (20)
gopher   Gopher (70)
hostname NIS hostname server (101)
ident    Ident Protocol (113)
irc      Internet Relay Chat (194)
kerberos Kerberos login (543)
kshell   Kerberos shell (544)
login    Login (rlogin, 515)
lpd      Printer service (315)
nntp     Network News Transport Protocol (119)
onmp-plain ONMP Cleartext (15001)
onmp-tls  ONMP TLS (15002)
p2p-auto-rp P2P Auto-RP (499)
pop2     Post Office Protocol v2 (109)
pop3     Post Office Protocol v3 (110)
smtp     Simple Mail Transport Protocol (25)
sunrpc   Sun Remote Procedure Call (111)
tacacs   TAC Access Control System (49)
talk     Talk (517)
telnet   Telnet (23)
time     Time (37)
uucp     Unix-to-Unix Copy Program (540)
whois    Nicname (43)
www      World Wide Web (HTTP, 80)
```

```
R1(config-std-nacl)#deny tcp any host 1.1.1.1 eq 80
```

→ Deny all packets destined for IP address 1.1.1.1/32, TCP port 80.

After the destination IP address and/or destination port numbers, there are many more options you can use to match (not necessary for the CCNA).

Some examples:

- **ack**: match the TCP ACK flag
- **fin**: match the TCP FIN flag
- **syn**: match the TCP SYN flag
- **ttl**: match packets with a specific TTL value
- **dscp**: match packets with a specific DSCP value

If you specify the protocol, source IP, source port, destination IP, destination port, etc, a packet must match ALL of those values to match the ACL entry. Even if it matches all except one of the parameters, the packet won't match that entry of the ACL.

1. Allow traffic from 10.0.0.0/16 to access the server at 2.2.2.2/32 using HTTPS.

```
R1(config-ext-nacl)#permit tcp 10.0.0.0 0.0.255.255 2.2.2.2 0.0.0.0 eq 443
```

2. Prevent all hosts using source UDP port numbers from 20000 to 30000 from accessing the server at 3.3.3.3/32.

```
R1(config-ext-nacl)#deny udp any range 20000 30000 host 3.3.3.3
```

3. Allow hosts in 172.16.1.0/24 using a TCP source port greater than 9999 to access all TCP ports on server 4.4.4.4/32 except port 23.

```
R1(config-ext-nacl)#permit tcp 172.16.1.0 0.0.0.255 gt 9999 host 4.4.4.4 neq 23
```



12. Important Port Numbers

TCP

Service	Port
FTP Data	20
FTP Control	21
SSH	22
Telnet	23
SMTP	25
HTTP	80

POP3	110
HTTPS	443

UDP

Service	Port
DHCP Server	67
DHCP Client	68
TFTP	69
SNMP Agent	161
SNMP Manager	162
Syslog	514

TCP + UDP

Service	Port
DNS	53

13. Example: Block HTTPS

Suppose:

PC network = 192.168.1.0/24

Server = 10.0.1.100

HTTPS = TCP 443

ACL:

```
R1(config)# ip access-list extended HTTP_SRV1
```

```
R1(config-ext-nacl)# deny tcp 192.168.1.0 0.0.0.255 host 10.0.1.100 eq 443
```

```
R1(config-ext-nacl)# permit ip any any
```

This means:

PCs in 192.168.1.0/24 cannot access 10.0.1.100 using HTTPS.

But other traffic is allowed because:

permit ip any any

14. Every Parameter Must Match

This is VERY important.

Suppose ACL says:

deny tcp 192.168.1.0 0.0.0.255 host 10.0.1.100 eq 443

A packet must match:

Protocol = TCP 
Source network = 192.168.1.0/24 
Destination = 10.0.1.100 
Destination port = 443 

Only when **ALL conditions match**, the ACL entry matches.

If even one condition is different:

Source 
Destination 
Protocol 
Port 

then that ACL entry does **not** match.

The router moves to the **next ACL entry**.

15. Where Should Extended ACL Be Applied?

A common CCNA rule:

Extended ACL → as close to the SOURCE as possible.

Why?

Suppose:

PC → R1 → R2 → Server

If traffic from PC must be blocked, block it at R1.

Otherwise, the unwanted packet may travel through:

PC → R1 → R2 → ... → ✗

That wastes bandwidth and router resources.

Standard ACL

Traditionally:

Standard ACL → close to the DESTINATION.

Because standard ACL only knows the source IP and may accidentally block other traffic from that source.

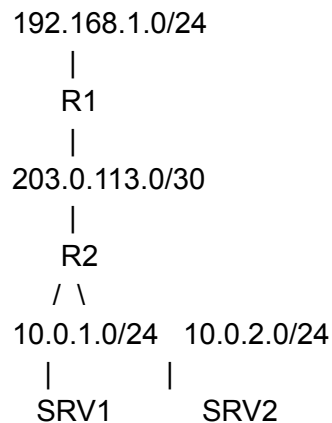
Memory trick

Extended → Source

Standard → Destination

16. Example Topology

From your screenshots:



Requirements:

Requirement 1

Hosts in:

192.168.1.0/24

cannot use HTTPS to:

SRV1 = 10.0.1.100

ACL:

```
deny tcp 192.168.1.0 0.0.0.255 host 10.0.1.100 eq 443
permit ip any any
```

Requirement 2

Hosts in:

192.168.2.0/24

cannot access:

10.0.2.0/24

ACL:

```
deny ip 192.168.2.0 0.0.0.255 10.0.2.0 0.0.0.255
permit ip any any
```

Requirement 3

Hosts in:

192.168.1.0/24

192.168.2.0/24

cannot ping:

10.0.1.0/24

10.0.2.0/24

Because ping uses **ICMP**:

```
deny icmp 192.168.1.0 0.0.0.255 10.0.1.0 0.0.0.255
deny icmp 192.168.1.0 0.0.0.255 10.0.2.0 0.0.0.255
deny icmp 192.168.2.0 0.0.0.255 10.0.1.0 0.0.0.255
permit ip any any
```

17. Applying ACL to Interface

Creating the ACL is **not enough**.

You must apply it to an interface.

Example:

```
R1(config)# interface g0/1
R1(config-if)# ip access-group HTTP_SRV1 in
```

Two directions:

in
out

in

Packet is filtered **when entering the interface**.

```
ip access-group ACL_NAME in
```

out

Packet is filtered **when leaving the interface**.

```
ip access-group ACL_NAME out
```

18. show access-lists

To see ACLs:

```
show access-lists
```

Example:

Extended IP access list HTTP_SRV1

```
10 deny tcp 192.168.1.0 0.0.0.255 host 10.0.1.100 eq 443
```

```
20 permit ip any any
```

The number:

10

20

is the **sequence number**.



19. ACL Processing Order

ACLs are processed:

TOP



BOTTOM

Example:

```
10 deny 192.168.1.10
```

```
20 permit 192.168.1.0/24
```

If source is:

192.168.1.10

Rule 10 matches first → **DENY**.

It never reaches rule 20.

Therefore:

ACL order matters.



20. Implicit Deny

Every ACL has an invisible rule at the bottom:

deny everything

Conceptually:

10 deny X
20 permit Y
30 permit Z

implicit deny

So if a packet doesn't match any rule:

 DENY

That's why we often add:

permit ip any any

if we want all other traffic to be allowed.

Final CCNA Memory Sheet

Standard ACL

Source IP only

1–99

1300–1999

Usually:

close to destination

Extended ACL

Source IP

Destination IP

Protocol

Source port

Destination port

100–199

2000–2699

Usually:

close to source

Important commands

```
ip access-list standard NAME
ip access-list extended NAME
show access-lists
show running-config | section access-list
ip access-group ACL_NAME in
ip access-group ACL_NAME out
ip access-list resequence ACL_ID START INCREMENT
```

Port operators

eq = equal
gt = greater than
lt = less than
neq = not equal
range = range of ports

Key protocol

TCP → reliable, connection-oriented
UDP → connectionless
ICMP → ping/error messages
IP → any IP protocol

Most important ACL logic

```
Packet
↓
Check Rule 10
↓
Match? → YES → permit/deny → STOP
↓ NO
Check Rule 20
↓
Match? → YES → permit/deny → STOP
↓ NO
Continue...
↓
Implicit DENY
```

The 4 things I would memorize for your CCNA exam:

1. Standard = source IP
2. Extended = source + destination + protocol + ports
3. ACL is checked top-to-bottom, first match wins
4. Extended close to source, Standard close to destination

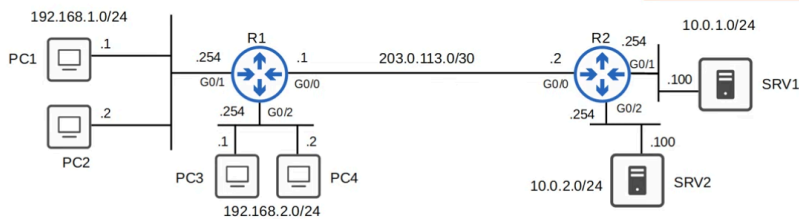


Extended ACLs

```
R1(config)#ip access-list extended BLOCK_10.0.2.0/24
R1(config-ext-nacl)#deny ip 192.168.2.0 0.0.0.255 10.0.2.0 0.0.0.255
R1(config-ext-nacl)#permit ip any any
R1(config-ext-nacl)#interface g0/2
R1(config-if)#ip access-group BLOCK_10.0.2.0/24 in
```

Requirements:

- Hosts in 192.168.1.0/24 can't use HTTPS to access SRV1.
- Hosts in 192.168.2.0/24 can't access 10.0.2.0/24.
- None of the hosts in 192.168.1.0/24 or 192.168.2.0/24 can ping 10.0.1.0/24 or 10.0.2.0/24.

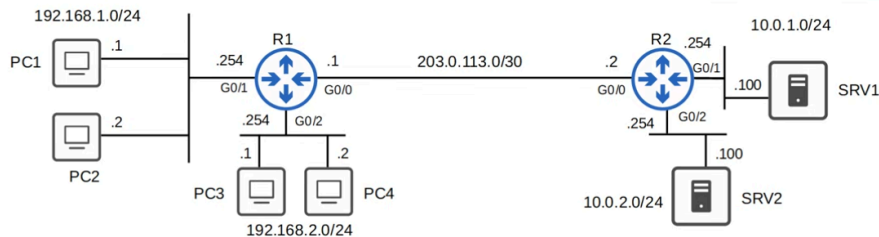


Extended ACLs

```
R1(config)#ip access-list extended BLOCK_ICMP
R1(config-ext-nacl)#deny icmp 192.168.1.0 0.0.0.255 10.0.1.0 0.0.0.255
R1(config-ext-nacl)#deny icmp 192.168.1.0 0.0.0.255 10.0.2.0 0.0.0.255
R1(config-ext-nacl)#deny icmp 192.168.2.0 0.0.0.255 10.0.1.0 0.0.0.255
R1(config-ext-nacl)#permit ip any any
R1(config-ext-nacl)#interface g0/0
R1(config-if)#ip access-group BLOCK_ICMP out
```

Requirements:

- Hosts in 192.168.1.0/24 can't use HTTPS to access SRV1.
- Hosts in 192.168.2.0/24 can't access 10.0.2.0/24.
- None of the hosts in 192.168.1.0/24 or 192.168.2.0/24 can ping 10.0.1.0/24 or 10.0.2.0/24.



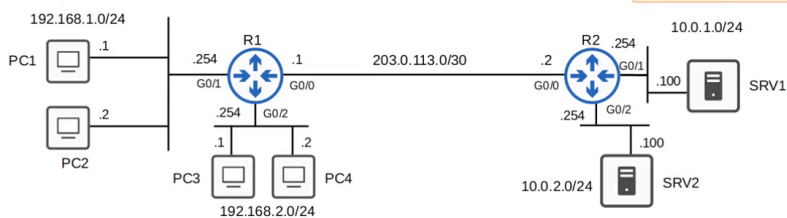


Extended ACLs

```
R1#show access-lists
Extended IP access list BLOCK_10.0.2.0/24
10 deny ip 192.168.2.0 0.0.0.255 10.0.2.0 0.0.0.255
20 permit ip any any
Extended IP access list BLOCK_ICMP
10 deny icmp 192.168.1.0 0.0.0.255 10.0.1.0 0.0.0.255
20 deny icmp 192.168.1.0 0.0.0.255 10.0.2.0 0.0.0.255
30 deny icmp 192.168.2.0 0.0.0.255 10.0.1.0 0.0.0.255
40 permit ip any any
Extended IP access list HTTP_SRV1
10 deny tcp 192.168.1.0 0.0.0.255 host 10.0.1.100 eq 443
20 permit ip any any
```

Requirements:

- Hosts in 192.168.1.0/24 can't use HTTPS to access SRV1.
- Hosts in 192.168.2.0/24 can't access 10.0.2.0/24.
- None of the hosts in 192.168.1.0/24 or 192.168.2.0/24 can ping 10.0.1.0/24 or 10.0.2.0/24.



Extended ACLs

```
R1#show ip interface g0/0
GigabitEthernet0/0 is up, line protocol is up
Internet address is 203.0.113.1/30
Broadcast address is 255.255.255.255
Address determined by non-volatile memory
MTU is 1500 bytes
Helper address is not set
Directed broadcast forwarding is disabled
Outgoing access list is BLOCK_ICMP
Inbound access list is not set
Proxy ARP is enabled
Local Proxy ARP is disabled
Security level is default
Split horizon is enabled
ICMP redirects are always sent
ICMP unreachable are always sent
ICMP mask replies are never sent
```

